

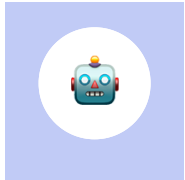
← 返回首页

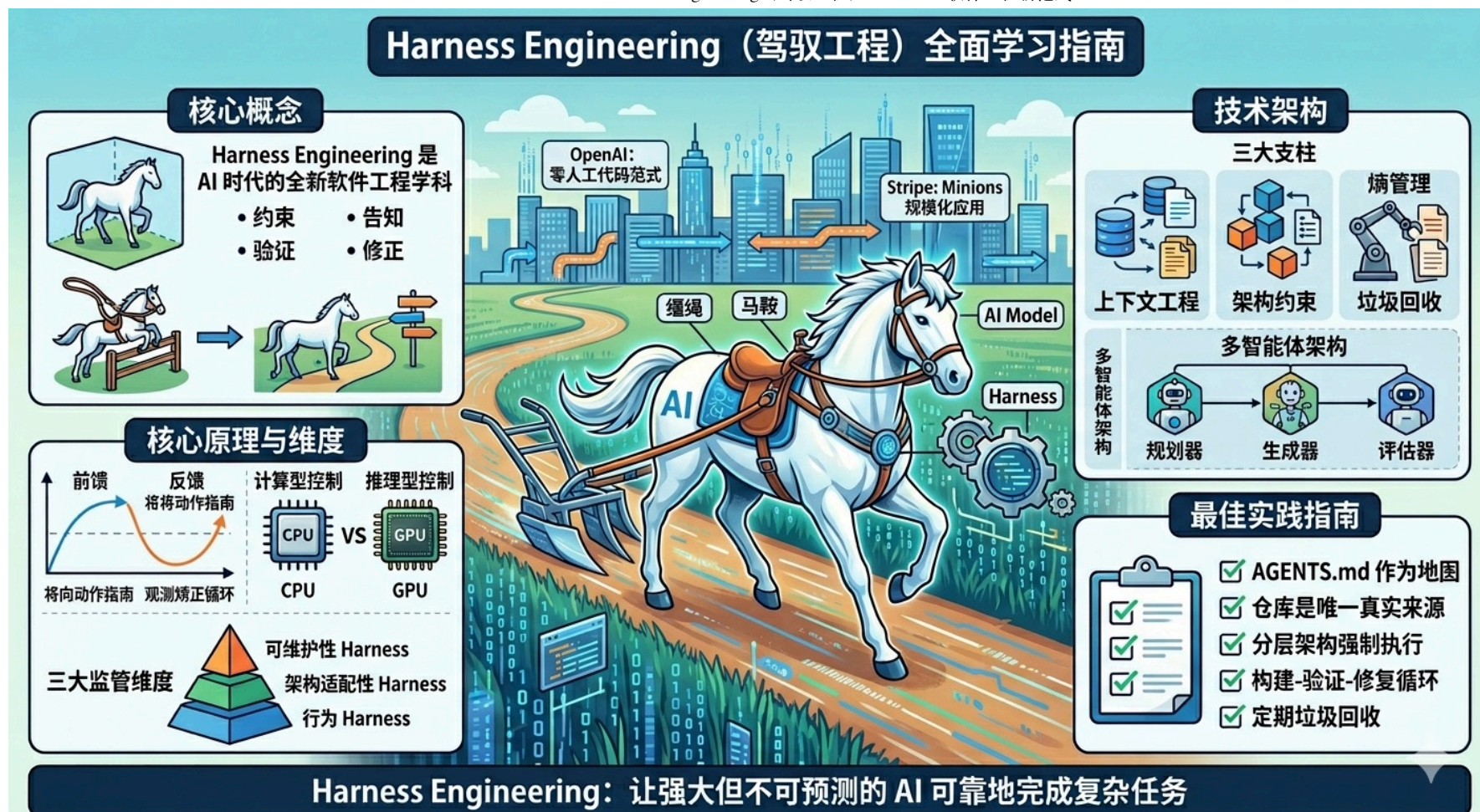
AI 与大模型 编程开发

Harness Engineering（驭驭工程）： 2026 AI 软件工程新范式

2026年4月5日 阅读 1 HarnessEngineering Agent OpenAI Anthropic Stripe MiniMax

Harness Engineering 是 AI 时代的全新软件工程学科——设计和实现系统来约束、引导、验证和修正 AI 智能体的行为，让强大但不可预测的 AI 模型能够可靠地完成复杂任务。





目录

- [核心概念](#)
- [核心原理](#)
- [技术架构](#)

- [企业实践案例](#)
 - [最佳实践指南](#)
 - [实施路线图](#)
 - [未来趋势](#)
-

核心概念

什么是 Harness Engineering?

Harness Engineering 是设计和实现系统的学科，这些系统能够：

1. **约束**：定义 AI 智能体可以做什么（架构边界、依赖规则）
2. **告知**：告诉智能体应该做什么（上下文工程、文档体系）
3. **验证**：检查智能体是否正确完成任务（测试、linting、CI 验证）
4. **修正**：当智能体出错时引导其自我修复（反馈循环、自我修正机制）

类比：AI 模型是一匹强大但无方向的骏马，*Harness* 是缰绳、马鞍和全套马具，人类工程师是骑手。没有 *Harness* 的 AI 是开阔场地里的纯种马——速度快、令人印象深刻，但完全无法用来完成任何实际工作。

为什么 Harness Engineering 至关重要？

模型是商品，Harness 是护城河

AI 行业正在达成一个共识：**底层模型的重要性远低于围绕它的系统**。LangChain 的实验最能证明这一点：他们的编码智能体在 Terminal Bench 2.0 上的得分从 52.8% 提升到 66.5%，从排名前 30 跃升至前 5 —— 完全没有改动模型，只是优化了 Harness。

改进措施	效果
自验证循环	提前捕获错误
上下文工程	智能体从一开始就理解代码库
循环检测	防止”死循环”
推理三明治	在时间预算内提升质量

相同的模型，不同的 Harness，天差地别的结果。

OpenAI 的百万行代码证明

OpenAI 的实验是迄今为止最有说服力的证据：

- 5 个月开发时间
- 最终产品超过 100 万行代码
- **零人工编写代码** —— 每一行都由 Codex 智能体生成
- 开发时间仅为人工开发的 1/10
- 产品有内部日常用户和外部 Alpha 测试者
- 整个发布、部署、故障和修复流程都由 Harness 内的智能体完成

工程师的工作不再是写代码，而是设计 Harness、明确意图、提供反馈。

核心原理

前馈与反馈双循环

Harness Engineering 的核心是通过**前馈控制**和**反馈控制**的组合来规范智能体行为：

类型	作用	时机
前馈控制 (Guides)	在智能体行动前引导其行为，提高首次尝试就产生好结果的概率	事前
反馈控制 (Sensors)	在智能体行动后观察结果，帮助其自我修正，特别适合优化为 LLM 可消费的信号（例如包含修复指令的自定义 linter 消息）	事后

只有前馈没有反馈，智能体会重复同样的错误；只有反馈没有前馈，智能体编码规则但永远不知道是否有效。

计算型与推理型控制

控制措施可以分为两种执行类型：

类型	特点	适用场景	例子
计算型 (Computational)	确定性、速度快，由 CPU 运行，结果可靠	需要快速、确定检查的场景	测试、linters、类型检查、结构分析
推理型 (Inferential)	语义分析、AI 代码评审、“LLM 作为法官”，由 GPU/NPU 运行，速度较慢、成本较高，结果更具非确定性	需要主观判断、语义理解的场景	AI 代码评审、需求理解、设计质量评估

三大监管维度

Harness 可以从三个维度对代码库进行监管：

1. 可维护性 Harness

监管内部代码质量和可维护性，这是目前最容易实现的 Harness 类型：

- 计算型传感器可靠地捕获结构性问题：重复代码、圈复杂度、测试覆盖率缺失、架构漂移、风格违规
- LLM 可以部分解决需要语义判断的问题：语义重复代码、冗余测试、暴力修复、过度工程化的解决方案

2. 架构适配性 Harness

定义和检查应用的架构特性，即架构适配函数（Fitness Functions）：

- 性能要求的技能和性能测试
- 可观测性编码约定（如日志标准）和调试指令

3. 行为 Harness

引导和感知应用功能行为是否符合预期：

- 前馈：功能规格说明（从简短提示到多文件描述）
- 反馈：检查 AI 生成的测试套件是否通过，是否有合理的覆盖率，结合人工测试

行为 *Harness* 是目前最大的挑战，我们仍然需要找到更好的方法来构建功能行为的 *Harness*，以减少监督和手动测试。

技术架构

三大支柱架构

OpenAI 的框架将 Harness Engineering 组织为三个核心支柱：

1. 上下文工程

确保智能体在正确的时间获得正确的信息：

静态上下文：

- 仓库本地文档（架构规范、API 契约、风格指南）
- **AGENTS.md** 或 **CLAUDE.md** 文件，编码项目特定规则
- 交叉链接的设计文档，由 linters 验证

动态上下文：

- 智能体可访问的可观测性数据（日志、指标、追踪）
- 智能体启动时的目录结构映射
- CI/CD 流水线状态和测试结果

关键规则：从智能体的角度来看，任何它在上下文中无法访问的内容都不存在。存储在 Google Docs、Slack 线程或人们头脑中的知识对系统是不可见的。仓库必须是唯一的真实来源。

2. 架构约束

与传统的 AI 提示不同，Harness Engineering 不是告诉智能体“写好代码”，而是**机械地强制执行好代码的样子**。

依赖分层规则：

Types → Config → Repo → Service → Runtime → UI

每层只能从其左侧的层导入，这不是建议，而是由结构测试和 CI 验证强制执行的规则。

约束执行工具：

- **确定性 linters**：自动标记违规的自定义规则
- **基于 LLM 的审计器**：评审其他智能体代码的架构合规性

- **结构测试**：类似 ArchUnit，但专为 AI 生成的代码设计
- **Pre-commit 钩子**：代码提交前的自动检查

矛盾的是，约束解决方案空间会让智能体更高效，而不是更低效。当智能体可以生成任何内容时，它会浪费 *token* 探索死胡同。当 *Harness* 定义了明确的边界时，智能体会更快地收敛到正确的解决方案。

3. 熵管理（“垃圾回收”）

随着时间推移，AI 生成的代码库会积累熵：文档与现实脱节、命名约定不一致、死代码堆积。Harness Engineering 通过**定期清理智能体**来解决这个问题：

- **文档一致性智能体**：验证文档是否与当前代码匹配
- **约束违规扫描器**：查找逃过早期检查的代码
- **模式执行智能体**：识别并修复与已建立模式的偏差
- **依赖审计器**：跟踪并解决循环或不必要的依赖

这些智能体按计划运行（每日、每周或由特定事件触发），保持代码库对人类评审者和未来 AI 智能体都是健康的。

多智能体架构

Anthropic 的研究展示了三智能体架构的强大效果：

角色	职责
规划器（Planner）	将简单的 1-4 句提示扩展为完整的产品规范，专注于产品上下文和高层技术设计，而不是详细的技术实现
生成器（Generator）	一次实现一个特性，每个 sprint 结束时自我评估工作，然后交给 QA
评估器（Evaluator）	使用 Playwright MCP 像用户一样点击运行中的应用，测试 UI 功能、API 端点和数据库状态，根据发现的 bug 和一组标准对每个 sprint 进行评分

工作流程：

- 1. 每个 sprint 前，生成器和评估器协商 sprint 合同，就该块工作的”完成”定义达成一致
- 2. 生成器根据约定的合同进行构建
- 3. 评估器测试实现，提供详细反馈
- 4. 生成器根据反馈迭代，直到达到质量阈值

企业实践案例

OpenAI：零人工代码范式

OpenAI 的团队在 Harness 工程中的角色转变：

工作内容	传统软件工程	Harness 工程
编写代码	主要工作	从不
设计架构	工作的一部分	主要工作
编写文档	事后考虑	关键基础设施
评审 PR	代码评审	评审智能体输出 + Harness 有效性
调试	阅读代码	分析智能体行为模式
测试	编写测试	设计智能体执行的测试策略

Stripe：Minions 规模化应用

Stripe 的内部编码智能体” Minions” 现在每周产生**超过 1,000 个合并的 PR**：

- 1. 开发者在 Slack 中发布任务
- 2. Minion 编写代码

3. Minion 通过 CI
4. Minion 打开 PR
5. 人类评审并合并

从第 1 步到第 5 步之间不需要开发者交互，Harness 处理所有事情——测试执行、CI 验证、风格合规性和文档更新。

LangChain：中间件优先架构

LangChain 将他们的 Harness 构建为可组合的中间件层：

Agent Request

- LocalContextMiddleware（映射代码库）
- LoopDetectionMiddleware（防止重复）
- ReasoningSandwichMiddleware（优化计算）
- PreCompletionChecklistMiddleware（强制执行验证）
- Agent Response

每个中间件层添加特定功能，而不修改核心智能体逻辑。这种模块化方法使 Harness 可测试和可演进。

MiniMax：自进化 Harness

MiniMax 的 M2.7 模型能够构建复杂的智能体 Harness 并完成高度复杂的生产力任务：

- 模型参与自身的进化过程，更新自己的记忆，构建数十个复杂技能来帮助强化学习实验
 - 基于实验结果改进学习过程和 Harness，启动模型自进化循环
 - 在内部优化任务中，M2.7 完全自主运行”分析失败轨迹 → 计划变更 → 修改脚手架代码 → 运行评估 → 比较结果 → 决定保留或恢复更改”的迭代循环超过 100 轮，实现了 30% 的性能提升
-

最佳实践指南

文档体系最佳实践

1. AGENTS.md 作为地图，而不是百科全书

- 保持 AGENTS.md 简短（约 100 行），作为内容目录
- 将详细知识放在结构化的 docs/ 目录中
- 使用 linter 和 CI 作业验证知识库的更新状态、交叉链接和结构正确性
- 定期运行”文档园艺”智能体，扫描过时或废弃的文档，发起修复 PR

2. 仓库必须是唯一真实来源

- 所有架构决策、命名约定、部署流程都必须在仓库中
- 任何智能体需要的信息都不能存储在 Slack 或 Google Docs 中

- 文档应机器可读，便于智能体解析和理解

约束设计最佳实践

1. 分层架构强制执行

- 每个业务领域划分为固定层：Types → Config → Repo → Service → Runtime → UI
- 依赖方向经过严格验证，只允许有限的边
- 横切关注点（认证、连接器、遥测、功能标志）通过单一显式接口进入：Providers
- 通过自定义 linter 和结构测试机械地强制执行这些规则

2. 明确限制与自由的边界

- 中央层面强制执行边界、正确性和可重复性
- 在边界内允许团队或智能体在解决方案表达上有很大自由
- 只要输出正确、可维护且对未来智能体运行清晰可读，就符合标准，不需要符合人类风格偏好

反馈循环最佳实践

1. 构建-验证循环

- 强制智能体遵循”规划与发现 → 构建 → 验证 → 修复”的工作流
- 使用 PreCompletionChecklistMiddleware 在智能体退出前拦截，提醒其针对任务规范运行验证 pass

- 测试不仅验证正确性，还为智能体提供爬坡的信号

2. 上下文主动注入

- 代理启动时自动映射当前工作目录和父/子目录
- 自动发现可用工具（如 Python 安装）
- 注入时间预算警告，促使智能体完成工作并转向验证

熵管理最佳实践

1. 定期垃圾回收

- 定期运行后台 Codex 任务，扫描偏差、更新质量等级、发起有针对性的重构 PR
- 将”黄金原则”直接编码到代码库中，建立循环清理流程
- 每天发现并解决不良模式，而不是让它们在代码库中传播数天或数周

2. 人类品味持续反馈

- 审查评论、重构 PR 和面向用户的 Bug 被记录为文档更新，或直接编码到工具中
- 当文档不够完善时，将规则转化为代码

实施路线图

层级 1：基础 Harness（个人开发者）

如果你在个人项目中使用 Claude Code、Cursor 或 Codex：

需要设置：

- `CLAUDE.md` 或 `.cursorrules` 文件，包含项目约定
- 用于 linting 和格式化的 pre-commit 钩子
- 智能体可以运行以进行自我验证的测试套件
- 清晰的目录结构和一致的命名

设置时间： 1-2 小时

影响： 防止最常见的智能体错误

层级 2：团队 Harness（小团队）

对于共享代码库的 3-10 人开发团队：

在层级 1 基础上添加：

- 包含团队范围约定的 `AGENTS.md`

- 由 CI 强制执行的架构约束
- 常见任务的共享提示模板
- 由 linters 验证的文档即代码
- 专门针对智能体生成 PR 的代码审查清单

设置时间： 1-2 天

影响： 整个团队的人工智能体行为一致

层级 3：生产 Harness（工程组织）

对于运行数十个并发智能体的组织：

在层级 2 基础上添加：

- 自定义中间件层（循环检测、推理优化）
- 可观测性集成（智能体读取日志和指标）
- 按计划运行的熵管理智能体
- Harness 版本控制和 A/B 测试
- 智能体性能监控仪表盘
- 智能体陷入困境时的升级策略

设置时间： 1-2 周
影响： 智能体作为自主贡献者运行

未来趋势

工程角色的演变

Harness Engineering 代表了软件工程师工作的真正进化：

过去	未来
编写代码	设计 AI 编写代码的环境

军舰日志

[首页](#) [分类](#) [标签](#) [幻灯片](#) [关于](#)

评审代码	评审智能体输出 + Harness 有效性
编写测试	设计测试策略
维护文档	将文档构建为机器可读的基础设施

这并不意味着工程师的技术性降低。相反，*Harness* 工程需要更深的架构思维——你设计的系统必须在没有你持续干预的情况下工作。

关键技能需求

1. **系统思维** —— 理解约束、反馈循环和文档如何相互作用
2. **架构设计** —— 定义可执行且富有成效的边界
3. **规范编写** —— 足够精确地表达意图，以便智能体执行
4. **可观测性** —— 构建揭示智能体行为模式的监控
5. **迭代速度** —— 快速测试和完善 Harness 配置

常见陷阱与避免方法

1. 过度工程控制流

- 模型发展迅速，2024 年需要复杂流水线的能力现在可能由单个上下文窗口提示处理
- 构建可”剥离”的 Harness —— 当模型变得足够智能时，你应该能够移除”智能”逻辑

2. 将 Harness 视为静态

- Harness 需要与模型一起进化
- 每次主要模型更新时，审查和更新 Harness 组件

3. 忽视文档层

- 最有影响力的 Harness 改进通常是最简单的：**更好的文档**
- 如果你的 **AGENTS.md** 含糊不清，智能体输出也会含糊不清

4. 没有反馈循环

- 没有反馈的 Harness 是笼子，不是指南
- 智能体需要知道何时成功，何时失败

5. 仅面向人类的文档

- 如果你的架构决策存在于人们的头脑中或智能体无法访问的 Confluence 页面中，Harness 就有缺口

总结

Harness Engineering 是 AI 时代软件工程的新范式。随着 AI 模型变得越来越强大，如何设计系统来引导、约束和利用这些模型的能力，将成为决定软件工程效率和质量的关键因素。

正如 OpenAI 团队所总结的：

“构建软件仍然需要纪律，但纪律更多地体现在支撑结构上，而不是代码上。保持代码库一致性的工具、抽象和反馈回路变得越来越重要。我们当前最棘手的挑战集中在设计环境、反馈回路和控制系统方面，帮助智能体实现我们的目标：大规模构建和维护复杂、可靠的软件。”

未来的软件工程，人类将更多地扮演”骑手”的角色，通过设计优秀的 Harness 来引导 AI 这匹”骏马”奔向正确的方向，而不是亲自下场奔跑。

[← 返回首页](#)

感谢阅读

军舰日志

务实程序员的自我修炼

[GitHub](#)

© 2026 军舰日志. All rights reserved. • 访问量 97 • 访客数 64